

Lucky 13; Atak na TLS 1.0-1.2 w trybie CBC

Olaf Surgut

4 grudnia 2024

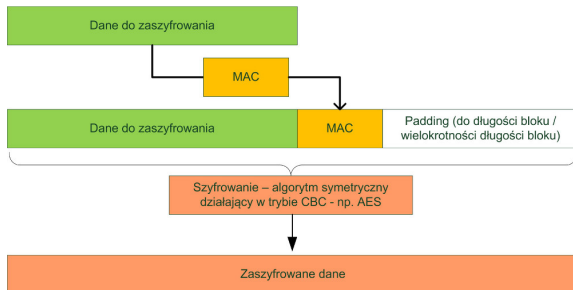
s2n-tls

- ▶ Skrót od "signal to noise"
- ▶ Nowa implementacja TLS 1.0-1.2 w trybie CBC od firmy Amazon
- ▶ ~ 6,000 linii kodu w C99 (vs. 70 000 w OpenSSL)
- ▶ Przed pierwszym wydaniem trzy niezależne zewnętrzne audyty oraz testy penetracyjne (Brak dodatkowych informacji od twórców na ten temat)

Gdzie tkwi szkopuł?

- ▶ **ZA**krótki kodzik budzi podejrzenia (OpenSSL potrzebuje 500 linii kodu na obronę przed atakiem)
- ▶ Atak będzie polegał na *Timing attack* podczas obliczania paddingu
- ▶ Różnice w obliczaniach poprawnego i niepoprawnego paddingu to mikrosekundy różnicy.
- ▶ Być może niemożliwe przez internet, ale gdy postawi się VM w data-center ofiary?

Trochę szczegółów



- ▶ Właściwe załatanie wymaga deterministycznego czasu obliczeń oraz stałej liczby zapytań pamięci
- ▶ To jest po prostu trudne

Co zrobiła nasza ofiara?

- ▶ Bezsensowne obliczania HMAC oraz sprawdzanie paddingu, aby zrekompensować czasy
- ▶ Dodatkowe losowe zatrzymania podczas błędu dekrypcji, aby zamaskować jakiegokolwiek pozostałości
- ▶ Ale...

Funkcja `s2n_verify_cbc`

```
67  int payload_and_padding_size = decrypted->size - mac_digest_size;
68
69  /* Determine what the padding length is */
70  uint8_t padding_length = decrypted->data[decrypted->size - 1];
71
72  int payload_length = payload_and_padding_size - padding_length \
    - 1;
73  if (payload_length < 0) {
74      payload_length = 0;
75  }
76
77  /* Update the MAC */
78  GUARD(s2n_hmac_update(hmac, decrypted->data, payload_length));
79  GUARD(s2n_hmac_copy(&copy, hmac));
80
81  /* Check the MAC */
82  uint8_t check_digest[S2N_MAX_DIGEST_LEN];
83  lte_check(mac_digest_size, sizeof(check_digest));
84  GUARD(s2n_hmac_digest(hmac, check_digest, mac_digest_size));
85
86  int mismatches = s2n_constant_time_equals(decrypted->data +
                                              payload_length,
                                              check_digest,
                                              mac_digest_size) ^ 1;
87
88  /* Compute a MAC on the rest of the data so that we perform
    the same number of hash operations */
89  GUARD(s2n_hmac_update(&copy, decrypted->data + payload_length +
                        mac_digest_size,
                        decrypted->size - payload_length -
                        mac_digest_size - 1));
```

- ▶ Istnieje ewidentna różnica, gdy ostatni bajt przekroczy 04.
- ▶ Dla naszego ataku dodatkowe wywołania obliczania HMAC zawsze będą mieć 60 bajtów.

Byte value	Cycles	Byte value	Cycles	Byte value	Cycles
0x00	2251.96	0x05	1746.49	...	...
0x01	2354.57	0x06	1747.65	0xfc	1640.79
0x02	2252.07	0x07	1705.62	0xfd	1634.61
0x03	2135.11	0x08	1808.73	0xfe	1648.70
0x04	2130.02	0x09	1806.50	0xff	1634.64

Table 3. Timing of function `s2n_verify_cbc` (in cycles) with $H = \text{SHA-256}$ for different values of last byte in the `decrypted` buffer, each cycle count averaged over 2^8 trials.

A co z losowymi zatrzymaniami?

```
s2n_recv.c
36  int s2n_read_full_record(struct s2n_connection *conn, \
                               uint8_t *record_type, int *isSSLv2)
97      /* Decrypt and parse the record */
98      if (s2n_record_parse(conn) < 0) {
99          GUARD(s2n_connection_wipe(conn));
100         if (conn->blinding == S2N_BUILT_IN_BLINDING) {
101             int delay;
102             GUARD(delay = s2n_connection_get_delay(conn));
103             GUARD(sleep(delay / 1000000));
104             GUARD(usleep(delay % 1000000));
105         }
106         return -1;
107     }
```

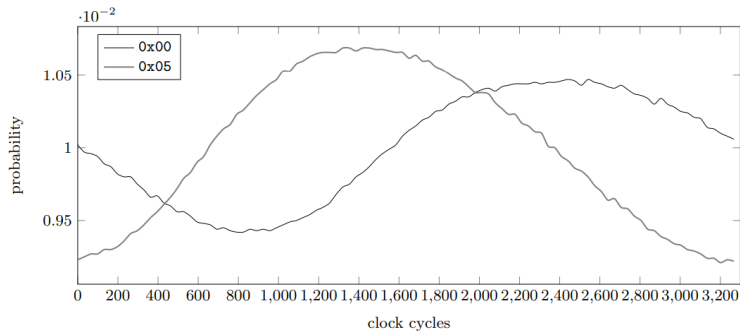


Fig. 8. Distribution of clock ticks modulo 3,300 for timing signals on Intel(R) Xeon(R) CPU E5-2667 v2 @ 3.30GHz with the maximum delay restricted to $d = 100,000$.

Dywergencja KL: 3.6×10^{-3}

Ograniczenia

- ▶ Atakujący musi być MITM
- ▶ Atakujący musi być blisko ofiary
- ▶ Oryginalny atak wymaga 2^{23} sesji
- ▶ Gdy obliczenie MAC się nie powiedzie to sesja jest zamykana.